

Estimating Unobservable Parameters for Offroad Autonomy

Micah Nye*, Nicholas Leone*, Nathan Ludlow*, Shubhra Aich*

Robotics Institute

Carnegie Mellon University

Pittsburgh, PA, USA

{micahn, nleone, nludlow, saich}@andrew.cmu.edu

Abstract—Off-road autonomous vehicles face significant challenges due to highly variable and difficult-to-model dynamics, including terrain-dependent slip, inconsistent traction, and changing vehicle configurations. While Model Predictive Path Integral (MPPI) control offers robustness to some model uncertainty, its effectiveness still depends heavily on the accuracy of the underlying dynamics model. In this work, we present a new control framework that combines MPPI with parameter estimation using an Unscented Kalman Filter (UKF). We model vehicle dynamics using a Dynamic Bicycle Model (DBM), which captures nonlinear effects such as lateral slip and steering response. The UKF estimates key parameters—including mass distribution, friction coefficients, and steering dynamics—based on logged state and control data. These parameters are then used to improve model rollouts in MPPI. We evaluate our approach by comparing the accuracy of trajectory predictions and control reconstruction across three models: a standard Kinematic Bicycle Model (KBM), an untrained DBM, and our UKF-trained DBM. Results show that our learned parameters DBM improves rollout fidelity and control accuracy, supporting more robust off-road planning and control.

Index Terms—Off-road Vehicles, Autonomy, Adaptive Control, Optimal Control

I. INTRODUCTION

Autonomous off-road driving presents a range of challenges that differ substantially from on-road navigation. Uneven terrain, inconsistent traction, and changing surface conditions introduce complex dynamics that make classical optimal control approaches difficult to apply. Additionally, factors such as payload variation and mechanical wear can alter internal vehicle parameters like steering response, throttle behavior, and center of mass. These variations degrade the performance of traditional control methods that rely on static models and assume consistent dynamics. To address this, Model Predictive Path Integral (MPPI) control has become a popular approach in off-road autonomy. MPPI provides robustness to model uncertainty by sampling multiple perturbed rollouts and computing a weighted average of the resulting control actions. This sampling-based approach has proven effective in off-road scenarios [1]–[5].

However, despite MPPI’s robustness, most implementations still rely on simple models like the Kinematic Bicycle Model

(KBM) for rollouts. This choice is often driven by computational constraints, as KBMs are lightweight and easy to tune. Yet in off-road environments, where slip, load transfer, and terrain variability are common, the simplifying assumptions of the KBM fail to capture critical dynamics. In contrast, the Dynamic Bicycle Model (DBM) includes effects like lateral tire slip and yaw behavior, making it more suitable for high-fidelity modeling—especially for vehicles with sufficient onboard computational resources.

While DBMs more accurately represent vehicle behavior, they introduce an added challenge: parameterization. Key parameters such as tire stiffness, friction coefficients, engine gain, and mass distribution are difficult to measure directly and often vary over time. Although MPPI can absorb some model error through its stochastic rollouts, significant mismatches still lead to degraded performance. This motivates the need for adaptive estimation techniques that can continuously refine model parameters during operation.

In this work, we address this challenge by introducing an adaptive control framework that integrates Unscented Kalman Filter (UKF)-based parameter estimation with DBM-based MPPI control. The UKF estimates critical dynamic parameters—such as the vehicle’s center of mass, moment of inertia, tire cornering stiffness, drag and friction coefficients, and steering dynamics—using logged state and control data. These parameters are stored in the vehicle model and used to generate more accurate rollouts in the MPPI controller.

By tightly coupling real-time parameter estimation with stochastic control, our method enables off-road vehicles to adapt to changing dynamics without manual tuning or offline calibration. This adaptive capability improves trajectory fidelity, enhances control performance, and provides a scalable foundation for autonomous operation in complex, variable environments.

II. RELATED WORKS

Adaptive control for ground vehicles has been studied extensively, particularly in the context of model uncertainty and terrain variability. Classical methods such as gain scheduling and adaptive linear controllers have been used to compensate for slowly varying dynamics, but they often require careful system identification and can struggle with fast-changing or nonlinear behaviors common in off-road settings. An alternate

*All authors contributed equally to this work.
Identify applicable funding agency here. If none, delete this.

old, but robust approach to these is to use state augmentation techniques with Kalman filters for real-time parameter estimation. In particular, the UKF has been used to estimate model parameters in a variety of robotic and automotive systems due to its ability to handle nonlinear dynamics without requiring explicit linearization.

The UKF is a nonlinear Bayesian filter that approximates the posterior distribution of system states using a deterministic sampling technique called the Unscented Transform [6]. To estimate model parameters, we augment the system state vector with parameters of interest and treat them as pseudo-states with their own dynamics (typically modeled as random walks). The UKF propagates a set of sigma points through the nonlinear dynamics, then computes the predicted mean and covariance. By updating this estimate using sensor observations, the UKF continuously refines the unknown parameters. [7]

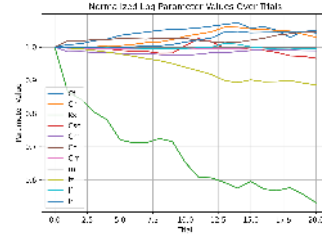
On the control side, sampling-based Model Predictive Control (MPC) methods such as MPPI control have shown strong performance in aggressive and uncertain environments for off-road vehicles. Originally formulated by Williams et al, MPPI avoids the need for gradient-based optimization by using stochastic trajectory rollouts to approximate the optimal control distribution [8]. Prior work has demonstrated the effectiveness of MPPI for off-road driving and agile robotic maneuvers, but it typically assumes access to accurate system dynamics [1]–[5], [9]. Our work bridges this gap by combining UKF-based parameter estimation with MPPI, enabling adaptive control in environments where the vehicle model is only partially known or subject to change.

III. METHODS

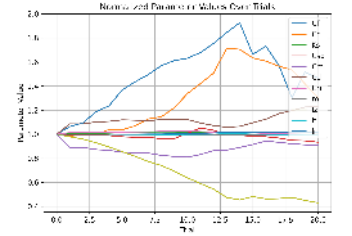
A. Overview

Our control architecture combines MPPI control with an offline parameter estimation via a UKF. The UKF estimates key dynamic parameters—such as friction coefficients and steering response—using a sequence of control inputs and corresponding observations in log-space. These estimated parameters are then used for the dynamics rollouts in the MPPI controller. MPPI samples control sequences, simulates the resulting state rollouts. These trajectory rollouts are evaluated on different objectives, and then importance sampling is applied to the actions, given a temperature value, λ , to yield the final control. This integration allows the control system to continuously adapt to evolving vehicle dynamics and environmental conditions in a data-driven way while reasoning about complex, non-linear cost functions.

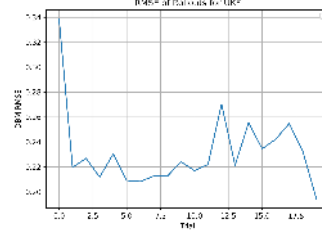
When running MMPI normally, we perform rollouts with a time horizon of 50. To tune the parameters for a particular initial state, we run the UKF offline for 20 iterations and run a rollout with a time horizon of 30 per iteration. For the first iteration, we start at the initial state and perform a rollout. We then compare the generated states from the rollout with a reference trajectory at the same timestamps of the generated states. For the second iteration, we start the rollouts at the state after the initial state. We keep running rollouts after shifting the initial states for each iteration. By performing rollouts with



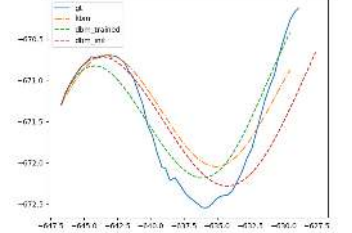
(a) Plot of log normalized parameters for each iteration



(b) Plot of normalized parameters for each iteration



(c) The RMSE of the UKF Rollouts for each iteration



(d) Rollouts with Time Horizon of 50 at Frame 200

a time horizon of 30 states and 20 iterations, we cover the 50 states that would be generated by a MPPI rollouts. By repeating this process offline for a number of rollouts we can generate the parameters for a model that is used in our MPPI rollouts.

Algorithm 1 UKF Parameter Estimation

- 1: Initialize UKF with model parameters $\hat{\theta}_0$ in log space
 - 2: Set time horizon $K = 20$
 - 3: **for** $t = 0$ to $K - 1$ **do**
 - 4: Observe current state x_t and action u_{t-1}
 - 5: Perform rollout with time horizon of 30 on (x_t, u_{t-1})
 - 6: Calculate residuals by comparing the rollout states to the reference trajectory
 - 7: **end for**
-

B. Dynamics Modeling

There are many variations of DBMs, each tailored to the available state measurements, specific vehicle configurations, and the parameters of interest. Typical DBMs for autonomous driving applications are single track, meaning each pair of forward/rear wheels are modeled as one cumulative wheel. Models can be double track as well where all four wheels are individually modeled. Prior work has explored a range of modeling approaches to capture the relevant dynamics of off-road vehicles [9]–[12]. In our work, we adopt a rear-wheel drive single track DBM that accounts for lateral slip dynamics at the tires. We use a linear tire model for simplicity, as opposed to a non-linear tire model such as Pajacka’s Model [13]. Additionally, we model our system’s steering dynamics to account for the vehicle’s nonlinear steering response. The derivation of the model used in our controller is presented below.

1) *Setup*: We use a single-track dynamics model as shown in Figure 2 for more accurate dynamics rollouts without requiring too many parameters to tune or measure. The state is decided as $x = [p_x \ p_y \ v_x \ v_y \ \theta \ \dot{\theta} \ \delta]^\top$ and control as $u = [\delta_C \ T]^\top$, where p is the position w.r.t. the odom frame, θ is the yaw, $\dot{\theta}$ is the yaw rate, v is robot velocity w.r.t the vehicle or body frame, δ is the current vehicle steering angle, δ_C is the commanded steering angle and T is the throttle.

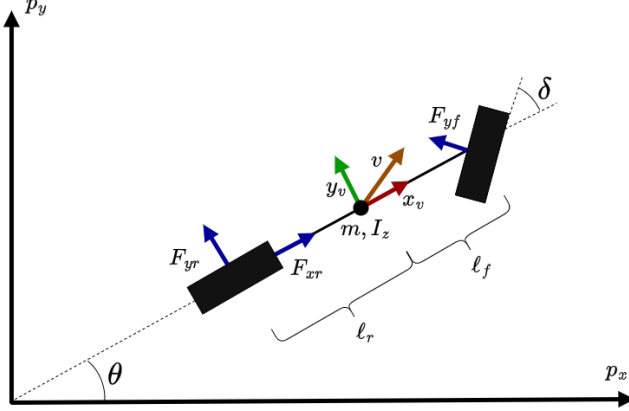


Fig. 2: Dynamic Bicycle Model Free Body Diagram

The equations of motion for the dynamics model are derived from Newton's Second Law, $F = ma$ when applied to the vehicle body. We assume the front tire is free rolling and has no longitudinal force:

$$\begin{aligned} ma_y &= F_y \\ ma_x &= F_x \\ I_z \ddot{\theta} &= M_z \end{aligned}$$

where the acceleration is given from the translational and centripetal accelerations in rigid body form: $a^b = \dot{v}^b + \dot{\theta}^b \times v^b$. This can be simplified to $a = \dot{v} + \dot{\theta}v$ since our model is 2D planar. Given the model, this can be:

$$\begin{aligned} m(\dot{v}_x - \dot{\theta}v_y) &= F_{xr} - F_{yf} \sin(\delta) \\ \dot{v}_x &= \frac{1}{m} (F_{xr} - F_{yf} \sin(\delta) + m\dot{\theta}v_y) \\ m(\dot{v}_y + \dot{\theta}v_x) &= F_{yr} + F_{yf} \cos(\delta) \\ \dot{v}_y &= \frac{1}{m} (F_{yr} + F_{yf} \cos(\delta) - m\dot{\theta}v_x) \\ I_z \ddot{\theta} &= l_f F_{yf} \cos(\delta) - l_r F_{yr} \\ \ddot{\theta} &= \frac{1}{I_z} (l_f F_{yf} \cos(\delta) - l_r F_{yr}) \end{aligned}$$

In addition to the dynamics for these 3 states, we derive the same for the other states. The position states are trivial to compute, and $\dot{\delta}$ is the steering nonlinearity we account for:

$$\begin{aligned} \dot{p}_x &= v_x \cos(\theta) - v_y \sin(\theta) \\ \dot{p}_y &= v_x \sin(\theta) + v_y \cos(\theta) \\ \dot{\delta} &= K_s \Delta\delta \cdot \mathcal{M}_{so}(\Delta\delta) C_{so} \end{aligned}$$

In summary, the dynamics for the state $x = [p_x \ p_y \ v_x \ v_y \ \theta \ \dot{\theta} \ \delta]^\top$ and control $u = [\delta_C \ T]^\top$ is as follows, requiring parameters: $m, I_z, \ell_f, \ell_r, K_s, \mathcal{M}_{so}, C_{so}$:

$$\begin{aligned} \dot{p}_x &= v_x \cos(\theta) - v_y \sin(\theta) \\ \dot{p}_y &= v_x \sin(\theta) + v_y \cos(\theta) \\ \dot{v}_x &= \frac{1}{m} (F_{xr} - F_{yf} \sin(\delta) + m\dot{\theta}v_y) \\ \dot{v}_y &= \frac{1}{m} (F_{yr} + F_{yf} \cos(\delta) - m\dot{\theta}v_x) \\ \ddot{\theta} &= \frac{1}{I_z} (l_f F_{yf} \cos(\delta) - l_r F_{yr}) \\ \dot{\delta} &= K_s \Delta\delta \cdot \mathcal{M}_{so}(\Delta\delta) C_{so} \end{aligned}$$

2) *Lateral Force Modeling*: Next, we express the lateral forces in terms of tire stiffness coefficients, using a linear tire model:

$$\begin{aligned} F_{yf} &= -C_{\alpha f} \alpha_f \\ F_{yr} &= -C_{\alpha r} \alpha_r \end{aligned}$$

Where α is the tire's side slip angle, expressed in terms of β , the vehicle body side slip, and δ , the steering angle. The tire sideslip for the rear tire does not consider the steering angle because the rear tires do not steer.

$$\begin{aligned} \alpha_f &= \beta_f - \delta \\ \alpha_r &= \beta_r \end{aligned}$$

The body side slip, β , is derived using the estimated lateral and longitudinal velocity from a SLAM solution, with $\beta = \tan^{-1} \left(\frac{v_y}{v_x} \right)$ for each wheel. Therefore:

$$\begin{aligned} \beta_f &= \tan^{-1} \left(\frac{v_{yf}}{v_{xf}} \right) \\ &= \tan^{-1} \left(\frac{v_y + l_f \dot{\theta}}{v_x} \right) \\ \beta_r &= \tan^{-1} \left(\frac{v_{yr}}{v_{xr}} \right) \\ &= \tan^{-1} \left(\frac{v_y - l_r \dot{\theta}}{v_x} \right) \end{aligned}$$

Putting it together, the lateral forces become:

$$F_{yf} = -C_{\alpha f} \alpha_f = -C_{\alpha f} \left(\tan^{-1} \left(\frac{v_y + l_f \dot{\theta}}{v_x} \right) - \delta \right)$$

$$F_{yr} = -C_{\alpha r} \alpha_r = -C_{\alpha r} \left(\tan^{-1} \left(\frac{v_y - l_r \dot{\theta}}{v_x} \right) \right)$$

3) *Longitudinal Force Modeling*: Obtaining the true longitudinal force on the vehicle is non-trivial. There are forces due to friction, drag, aerodynamics, and engine torque. To approximate this force, we will use the following model with C_m as a linear engine coefficient, applied to the engine throttle, C_d as the drag coefficient of the vehicle, and C_{rr} as the tire rolling resistance coefficient, similar to kinetic friction.

$$F_{xr} = F_{engine} - F_{drag} - F_{rolling_resistance}$$

$$F_{xr} = C_m T - C_d v_x^2 - C_{rr} N$$

4) *Full Dynamic Bicycle Model*: In summary, the full DBM dynamics for the state $x = [p_x \ p_y \ v_x \ v_y \ \theta \ \dot{\theta} \ \delta]^T$ and control $u = [\delta_C \ T]^T$ is as follows, requiring parameters: $m, I_z, \ell_f, \ell_r, C_{\alpha f}, C_{\alpha r}, C_m, C_d, C_{rr}, K_s, \mathcal{M}_{so}, C_{so}$:

$$\begin{bmatrix} \dot{x}_w \\ \dot{y}_w \\ \dot{\theta} \\ \dot{v}_x \\ \dot{v}_y \\ \ddot{\theta} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} v_x \cos(\theta) - v_y \sin(\theta) \\ v_x \sin(\theta) + v_y \cos(\theta) \\ \dot{\theta} \\ \frac{1}{m} (F_{xr} - F_{yf} \sin \delta + m \dot{\theta} v_y) \\ \frac{1}{m} (F_{yr} + F_{yf} \cos \delta - m \dot{\theta} v_x) \\ \frac{1}{I_z} (\ell_f F_{yf} \cos \delta - \ell_r F_{yr}) \\ K_s \Delta \delta \cdot \mathcal{M}_{so}(\Delta \delta) C_{so} \end{bmatrix}$$

Where:

$$F_{yf} = -C_{\alpha f} \alpha_f = -C_{\alpha f} \left(\tan^{-1} \left(\frac{v_y + l_f \dot{\theta}}{v_x} \right) - \delta \right)$$

$$F_{yr} = -C_{\alpha r} \alpha_r = -C_{\alpha r} \left(\tan^{-1} \left(\frac{v_y - l_r \dot{\theta}}{v_x} \right) \right)$$

C. MPPI Control

We use MPPI, a sampling-based stochastic optimal control method derived from path integral formulations of stochastic control problems. At each control step, MPPI samples a set of control sequences from a distribution (usually Gaussian) and simulates the resulting trajectories forward using a dynamics model. Each trajectory is assigned a cost based on task-specific objectives (e.g., tracking error, control effort). A softmax-weighted average of the sampled control sequences is computed to form the next optimal control input. MPPI is particularly well-suited for nonlinear, non-convex problems and

supports real-time implementation through parallel sampling and model rollouts.

With the dynamics derived, we now can run an MPPI solution online to solve for the optimal control solution. We have an initial action library to sample from, and we apply a single-step gaussian over the previous optimal control. Then we roll out all these actions using parallel processing and retrieve a set of trajectories. These trajectories evaluated given a cost function. The cost function we use it made up of many terms: a traversability map, a speed map, a euclidean distance to goal, and a control effort term. The cost function is given as:

$$J_{MPPI} = J_{trav_map} + J_{speed_map} + J_{goal}$$

The traversability and speed maps are learned ego-centric maps using MaxEntIRL, an inverse reinforcement learning technique using expert demonstrations to inform costs on your environment using prior work [14]. The input to these models are Birds-eye-view (BEV) feature maps with features from a visual foundation model, namely Radio or DINOv2 [15], [16] using prior work [17]. The distance to goal cost and control effort costs are trivial costs.

D. Parameter Estimation

For the prediction step, we generate our predicted sigma points. Then, we estimate mean and covariance matrix with the sigma points.

UKF Prediction

- 1) $\chi_{t-1} = (x_{t-1} \quad x_{t-1} \pm \sqrt{(n+\lambda)\Sigma_{t-1}})$
- 2) $\bar{\mu}_t = \sum_{i=0}^{2n} w_m^i \chi_{t-1}^i$
- 3) $\bar{\Sigma}_t = \sum_{i=0}^{2n} w_c^i (\chi_{t-1}^i - \bar{\mu}_t)(\chi_{t-1}^i - \bar{\mu}_t)^T + R_t$

For the correction step, we generate new sigma points using the estimated mean instead of the previous state x_{t-1} . We then transform our sigma points from the model parameters to the model states. We then correct our model parameters and model covariance matrix with the residuals of the estimated model state and reference states.

UKF Correction

- 1) $\bar{\chi}_t = (\bar{\mu}_t \quad \bar{\mu}_t \pm \sqrt{(n+\lambda)\bar{\Sigma}_{t-1}})$
- 2) $\bar{Z}_t = h(\bar{\chi}_t)$
- 3) $\hat{z}_t = \sum_{i=0}^{2n} w_m^i \bar{Z}_t^i$
- 4) $S_t = \sum_{i=0}^{2n} w_c^i (\bar{Z}_t^i - \hat{z}_t)(\bar{Z}_t^i - \hat{z}_t)^T + Q_t$
- 5) $\bar{\Sigma}_t^{x,z} = \sum_{i=0}^{2n} w_c^i (\bar{\chi}_t^i - \hat{\mu}_t)(\bar{Z}_t^i - \hat{z}_t)^T$
- 6) $K_t = \bar{\Sigma}_t^{x,z} S_t^{-1}$
- 7) $x_t = \bar{\mu}_t + K_t(z_t - \hat{z}_t)$

$$8) \Sigma_t = \bar{\Sigma}_t - K_t S_t K_t^T$$

x_t : Dynamic vehicle parameters

Σ_t : Covariance of the parameters

R_t : Measurement noise

Q_t : Process noise

$h(x)$: DBM Rollout

To determine what parameters were most important to estimate we ran a parameter sweep, where we varied each parameters value and observed its overall impact to the models output. The results of this parameter sweep can be seen in Figure 3, and show that the parameters that are most important to learn are those that change the output of the model drastically with only small variations to the parameter itself. This includes the parameters: Cd , Cf , Cm , Cr , lf , and lr . The UKF was then designed such that the noise and measurement coefficients were set to values that allowed these parameters to better fit to the ground truth trajectory.

IV. RESULTS

We evaluate the effectiveness of our proposed MPPI controller using our UKF-trained DBM by comparing it against alternative vehicle modeling approaches. Specifically, we compare our trained DBM to two baselines: a standard KBM which was previously used for control on this vehicle, and an untrained DBM using nominal, pre-measured parameters for the vehicle. Due to hardware limitations, we were unable to deploy the UKF-trained DBM in real-time on the physical vehicle. Additionally, the lack of suitable off-road simulation platforms limited our ability to evaluate the system in a fully closed-loop simulation.

Despite these constraints, we demonstrate the capability of our adaptive model to support robust and accurate control through two key evaluations. First, we assess trajectory roll-out accuracy by comparing how well each model replicates recorded vehicle trajectories when given the same sequence of control inputs. This allows us to quantify how closely each model captures real-world vehicle dynamics. Second, we evaluate control command consistency by comparing the control actions produced by the MPPI controller under each model against the actual control commands logged during real-world driving. This test highlights each model's ability to reproduce realistic and feasible controls given the observed system state.

The rollout examples shown in Figure 4 highlight the qualitative differences between the three vehicle models when tracking a dynamic trajectory. These rollouts were generated from a slalom maneuver executed on uneven terrain, with the vehicle's steering modulated by a sine wave input. This creates a challenging scenario in which the vehicle experiences lateral slip and follows uncertain paths due to surface irregularities. While the KBM captures the general motion trend, it provides the least accurate representation of the vehicle's true dynamics. The untrained DBM offers improved tracking, particularly during turning, where slip effects are more pronounced. However, the UKF-trained DBM achieves the highest fidelity, accurately

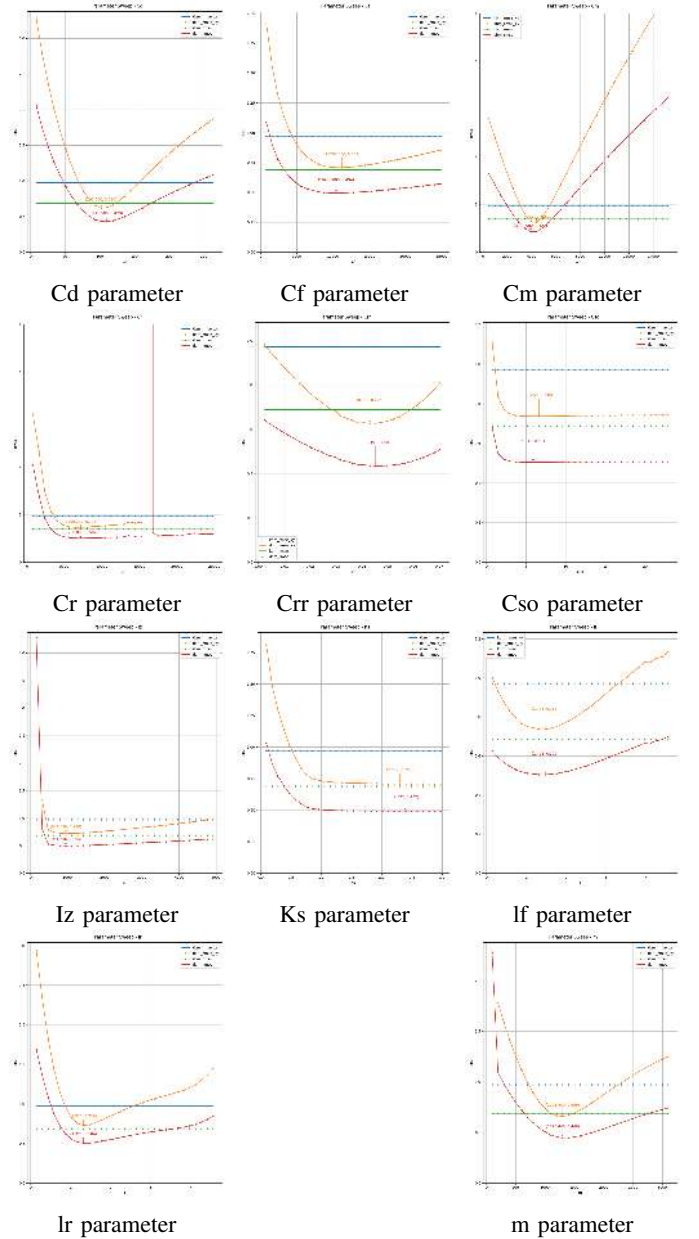


Fig. 3: This shows the effects of different parameters on the resulting output of the DBM model as they vary. This information is used to determine what parameters are most important for our model to learn.

reproducing the vehicle's trajectory and capturing complex nonlinear behaviors, making it the most effective of the three models.

These qualitative findings are further supported by the quantitative results in Table I, where we evaluate average RMSE across multiple recorded rollout trajectories. The trained DBM consistently achieves the lowest error relative to ground truth, confirming its superior ability to model the vehicle's dynamics under realistic off-road conditions.

We also evaluate each model's ability to reproduce the

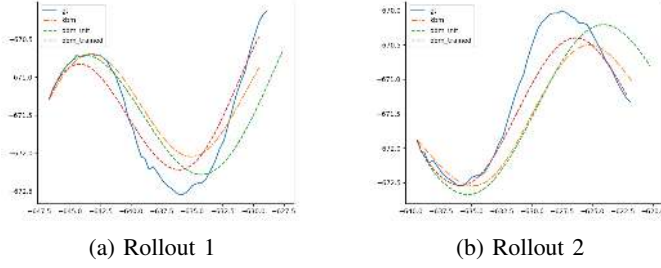


Fig. 4: This shows the effects of different parameters on the resulting output of the DBM model as they vary. This information is used to determine what parameters are most important for our model to learn.

TABLE I Rollout RMSE comparison. For both the full-state and only X-Y state, the UKF-Trained DBM has the lowest RMSE.

State RMSE			
State	KBM	Untrained DBM	Trained DBM
Full State	0.5575	0.4967	0.4139
XY Only	0.7870	0.7221	0.5960

vehicle’s actual control commands when provided with the same goal trajectory and observed vehicle states. The results of this analysis are shown in Figure 5 and Table II. While the control outputs from all models appear broadly similar, the trained DBM most closely matches the sine wave pattern of the vehicle’s true control commands. All models exhibit some level of control noise due to the stochastic nature of MPPI sampling, resulting in unsmooth trajectories. However, the quantitative results clearly show that the UKF-trained DBM yields the lowest control RMSE, indicating its superior ability to replicate the underlying vehicle dynamics. This improved model fidelity directly contributes to more accurate and effective control performance.

TABLE II Control RMSE comparison. We evaluate RMSE of each individual control and the full control space. KBM better matches the throttle, but the trained DBM better matches Steering.

Control RMSE			
Control	KBM	Untrained DBM	Trained DBM
Steering	0.1883	0.1597	0.1372
Throttle	0.1713	0.2384	0.2229
Full	0.1800	0.2029	0.1851

Finally, we share our results of the ego going to a goal position with the full stack running online in Figure Figure 6. The maps are from learned costmaps generated online as explained in the method section, and we run MPPI with the trained DBM to see how the agent can track a goal position.

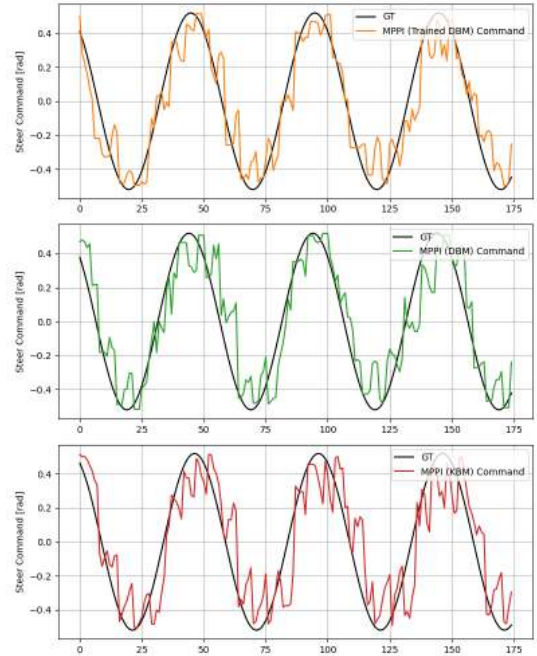


Fig. 5: Two sample hardware rollouts for our off-road vehicle, with different trajectory estimates for our different models given an initial state, and control inputs

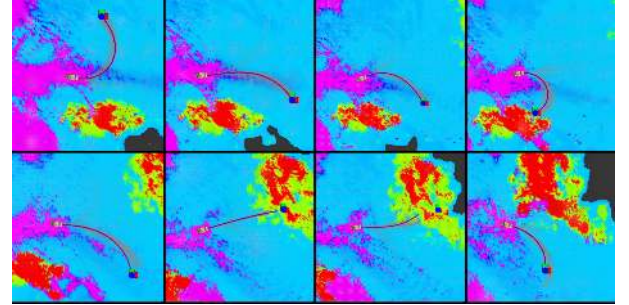


Fig. 6: Full autonomy execution of the system with learned costmaps for MPPI to use with a trained DBM.

V. FUTURE WORK

In future work, we plan to expand on our approach in two key directions. First, we aim to deploy our MPPI controller using the offline-trained Dynamic Bicycle Model (DBM) to evaluate system performance in real-world conditions. Second, we intend to implement real-time parameter estimation on the vehicle using our UKF-based method, enabling the model to adapt to dynamic changes in vehicle behavior. These extensions will allow us to assess the system’s robustness and effectiveness through on-vehicle testing in off-road environments.

The parameter estimation in real time would follow the algorithm shown in algorithm 1.

REFERENCES

- [1] M. Gandhi, B. Vlahov, J. Gibson, G. Williams, and E. A. Theodorou, "Robust Model Predictive Path Integral Control: Analysis and Performance Guarantees," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1423–1430, Apr. 2021, arXiv:2102.09027 [eess]. [Online]. Available: <http://arxiv.org/abs/2102.09027>
- [2] T. Han, A. Liu, A. Li, A. Spitzer, G. Shi, and B. Boots, "Model Predictive Control for Aggressive Driving Over Uneven Terrain," Jun. 2024, arXiv:2311.12284 [cs]. [Online]. Available: <http://arxiv.org/abs/2311.12284>
- [3] H. Lee, T. Kim, J. Mun, and W. Lee, "Learning Terrain-Aware Kinodynamic Model for Autonomous Off-Road Rally Driving With Model Predictive Path Integral Control," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7663–7670, Nov. 2023, arXiv:2305.00676 [cs]. [Online]. Available: <http://arxiv.org/abs/2305.00676>
- [4] J. Yang, M. Kang, S. Lee, and S. Kim, "Hybrid A*-Guided Model Predictive Path Integral Control for Robust Navigation in Rough Terrains," *Mathematics*, vol. 13, no. 5, p. 810, Jan. 2025, number: 5 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: <https://www.mdpi.com/2227-7390/13/5/810>
- [5] V. Mundheda, Z. Wu, and J. Schneider, "Towards Off-Road Autonomous Driving via Planner Guided Policy Optimization," Oct. 2024. [Online]. Available: <https://openreview.net/forum?id=uaKBM9sGEm>
- [6] S. J. Julier and J. K. Uhlman, "A New Extension of the Kalman Filter to Nonlinear Systems," *Proceedings of SPIE*, vol. 3068, no. VI, pp. 182–193, 1997.
- [7] P. S. Maybeck, *Stochastic models, estimation and control*, ser. Mathematics in science and engineering. New York: Academic Press, 1979, no. vol. 141.
- [8] G. Williams, A. Aldrich, and E. A. Theodorou, "Model Predictive Path Integral Control: From Theory to Parallel Computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, Feb. 2017. [Online]. Available: <https://arc.aiaa.org/doi/10.2514/1.G001921>
- [9] A. Raji, A. Liniger, A. Giove, A. Toschi, N. Musiu, D. Morra, M. Verucchi, D. Caporale, and M. Bertogna, "Motion Planning and Control for Multi Vehicle Autonomous Racing at High Speeds," in *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, Oct. 2022, pp. 2775–2782, arXiv:2207.11136 [cs]. [Online]. Available: <http://arxiv.org/abs/2207.11136>
- [10] A. Raji, N. Musiu, A. Toschi, F. Prignoli, E. Mascaro, P. Musso, F. Amerotti, A. Liniger, S. Sorrentino, and M. Bertogna, "A Tricycle Model to Accurately Control an Autonomous Racecar with Locked Differential," in *2023 IEEE 11th International Conference on Systems and Control (ICSC)*, Dec. 2023, pp. 782–789, arXiv:2312.14808 [eess]. [Online]. Available: <http://arxiv.org/abs/2312.14808>
- [11] J. Mai, "System Design, Modelling, and Control for an Off-Road Autonomous Ground Vehicle."
- [12] D. Kalaria, H. Xue, W. Xiao, T. Tao, G. Shi, and J. M. Dolan, "Agile Mobility with Rapid Online Adaptation via Meta-learning and Uncertainty-aware MPPI," Mar. 2025, arXiv:2410.06565 [cs]. [Online]. Available: <http://arxiv.org/abs/2410.06565>
- [13] H. B. Pacejka and E. Bakker, "THE MAGIC FORMULA TYRE MODEL," *Vehicle System Dynamics*, vol. 21, no. sup001, pp. 1–18, Jan. 1992. [Online]. Available: <http://www.tandfonline.com/doi/abs/10.1080/00423119208969994>
- [14] S. Triest, M. G. Castro, P. Maheshwari, M. Sivaprakasam, W. Wang, and S. Scherer, "Learning risk-aware costmaps via inverse reinforcement learning for off-road navigation," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 924–930.
- [15] M. Ranzinger, G. Heinrich, J. Kautz, and P. Molchanov, "Am-radio: Agglomerative vision foundation model reduce all domains into one," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2024, pp. 12 490–12 500.
- [16] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "Dinov2: Learning robust visual features without supervision," 2024. [Online]. Available: <https://arxiv.org/abs/2304.07193>
- [17] S. Triest, M. Sivaprakasam, S. Aich, D. Fan, W. Wang, and S. Scherer, "Velociraptor: Leveraging visual foundation models for label-free, risk-aware off-road navigation," in *8th Annual Conference on Robot*

Learning, 2024. [Online]. Available: <https://openreview.net/forum?id=AhEE5wrcLU>